

Introducción a JAVASCRIPT

N A N C Y B E R N A L



INTRODUCCIÓN

En esta unidad veremos REACT, pero ¿qué temas son los que tienes que saber para poder dominar y comenzar con React?

- Let y const para crear variables Promises, los módulos de script
- APIs de JavaScript, eventos, Array y Object methods.
- Async await DOM y Fetch API.

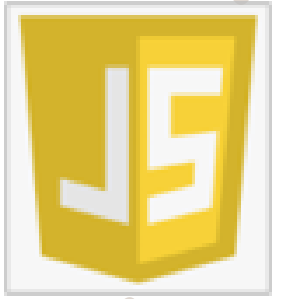


CONTENIDOS

- **¿Cuánto JavaScript debo saber para trabajar con REACT?**



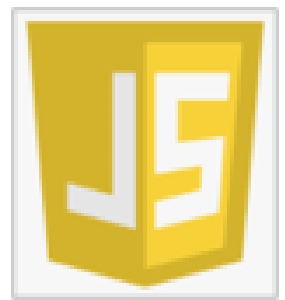
¿Cuánto JavaScript debo saber para trabajar con REACT?



- React es una librería basada en JavaScript.
- Mientras más sepas de JavaScript, más fácil va a ser aprender React.
- En el curso utilizaremos TypeScript, pero recuerda, TypeScript se compila a JavaScript y lo mismo que tienes en JavaScript funciona en TypeScript.
- Hoy en día JavaScript saca una versión nueva cada año.
- Algo muy importante es que las nuevas versiones de JavaScript que salen cada año agregan cuatro o cinco funciones nuevas.



CONCEPTOS BÁSICOS DE JS:



VARIABLES

- JavaScript te permite crear variables de **tres formas** utilizando:

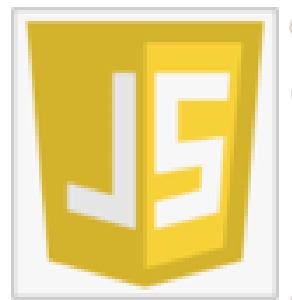
- ✓ **var**
- ✓ **let**
- ✓ **const**

- Veamos las diferencias entre cada una de ellas:

- ✓ **var** hoy en día casi no se utiliza.



CONCEPTOS BÁSICOS DE JS:

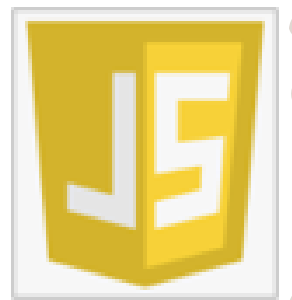


VARIABLES

- **let:**
 - ✓ **let nombreVariable** [no puede iniciar con n° y no tiene tipo].
 - ✓ Declaración de dos formas:
 - Declaración sola **let nombre**
 - Declaración e inicialización **let edad = 10**
 - ✓ Su valor puede reasignarse tantas veces como sea necesario [incluso puede variar su tipo]



CONCEPTOS BÁSICOS DE JS:

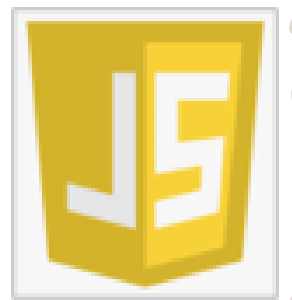


VARIABLES

- **const:**
 - ✓ **const nombre= "Bea"** [este valor es constante].
 - ✓ **const nombre** sin inicializar da error
 - ✓ El valor de una const NO PUEDE ser REASIGNADO.



TTIPOS DE DATOS:



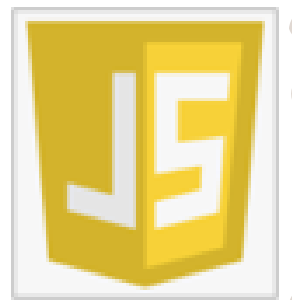
Tipos de Datos:

- **Undefined:**
 - ✓ Cuando declaras tu variable y no la inicializas.
- **String ó Cadenas de Texto:**
 - ✓ **let o const direccion = "Mi casa"**
 - ✓ **Puedes utilizar comillas simples o dobles**

****Los tipos pueden revisarse con typeof**



TIPOS DE DATOS:



Tipos de Datos:

- **number:** todos son números, no hay diferencia entre enteros/decimales, negativos ni positivos.

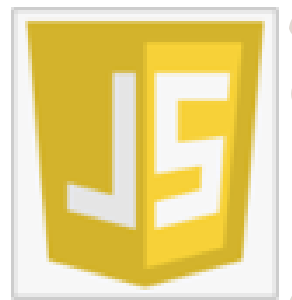
```
//Numbers
const numero1 = 34.3
const numero2 = 2
const numero3 = -78
```

- **BigInt:** se usa poco y es para valores muy grandes.

```
const nroGrande = BigInt(12312323123124345546867789999)
console.log(nroGrande)
console.log(typeof nroGrande)
```



TIPOS DE DATOS:

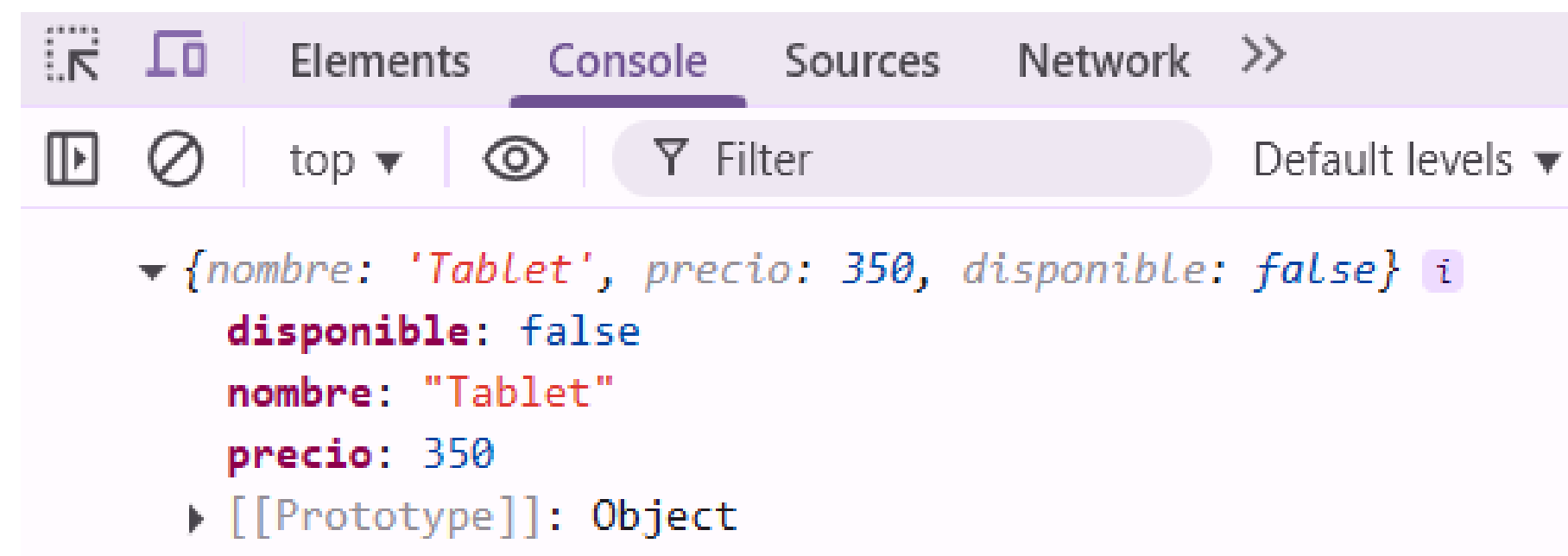


Tipos de Datos:

- **Object**

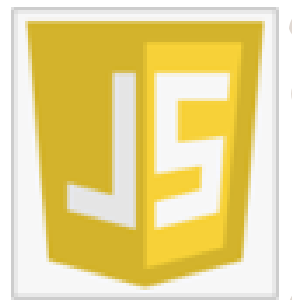
```
//Objetos
const producto = {
  nombre : 'Tablet',
  precio : 350,
  disponible : false
}
```

```
console.log(producto)
```





TIPOS DE DATOS:



Tipos de Datos:

- **Object**

```
console.log(typeof producto)
```

object

js04.js:10

```
console.table(producto)
```

js04.js:13

(index)	Value
nombre	'Tablet'
precio	350
disponible	false

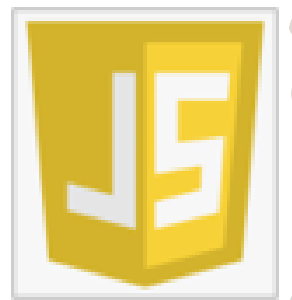
► Object

```
console.log(producto.precio)
```

350



TIPOS DE DATOS:



Tipos de Datos:

- **Objec**, en versiones anteriores a 6 también se utilizaba:

```
const nomProd = producto.nombre  
console.log(nomProd)
```

Tablet

[js04.js:20](#)

- **Destructuring**: es mas eficiente que hacerlo uno a uno como arriba

```
const { nombre, precio, disponible } = producto  
console.log(nombre)  
console.log(precio)  
console.log(disponible)
```

Tablet

[js04.js:29](#)

350

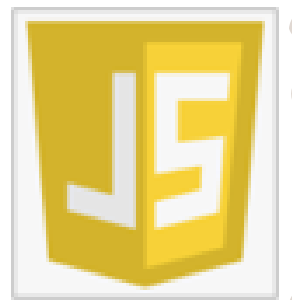
[js04.js:30](#)

false

[js04.js:31](#)



TIPOS DE DATOS:



Tipos de Datos:

- **Objec Literal Enhacement:** Esto funciona para una copia literal, es decir los campos de destino mantienen el mismo nombre que en su origen.

```
//Objec Literal Enhacement
const autenticado = true
const usuario = 'Lulú'

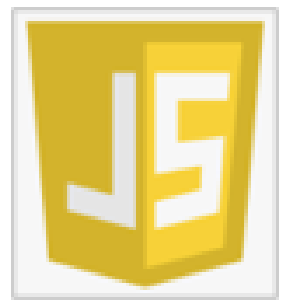
const nuevoObjeto = {
  autenticado,
  usuario
}

console.log(nuevoObjeto)
```

► `{autenticado: true, usuario: 'Lulú'}`



TIPOS DE DATOS:



Manipulación de Objetos

```
//Objetos - Manipulación
const producto = {
  nombre : 'Laptop',
  precio : 800,
  disponible : false
}

console.log(producto)

//Reescribir un valor
producto.disponible = true

//Sino existe, lo va a añadir
producto.imagen = 'imagen.jpg'

//Eliminar propiedad
delete producto.precio

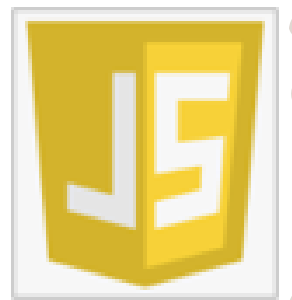
console.log(producto)
```

► {nombre: 'Laptop', precio: 800, disponible: false}

► {nombre: 'Laptop', disponible: true, imagen: 'imagen.jpg'}



TIPOS DE DATOS:



Manipulación de Objetos - freeze: Hacer un objeto inmodificable

```
//Objetos - Manipulación
const producto = {
  nombre : 'Laptop',
  precio : 800,
  disponible : false
}

console.log(producto)

//Hacerlo inmodificable
Object.freeze(producto)

//Reescribir un valor
producto.disponible = true

//Sino existe, lo va a añadir
producto.imagen = 'imagen.jpg'

//Eliminar propiedad
delete producto.precio

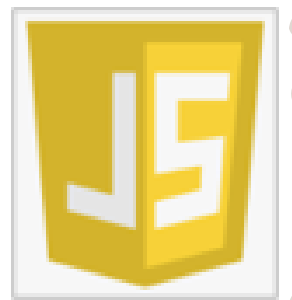
console.log(producto)
```

► {nombre: 'Laptop', precio: 800, disponible: false}

► {nombre: 'Laptop', precio: 800, disponible: false}



TIPOS DE DATOS:



```
1 //Objetos - Manipulación
2 const producto = {
3   nombre : 'Laptop',
4   precio : 800,
5   disponible : false
6 }
7
8 console.log(producto)
9
10 //Modificable - sin ingresos nuevos
11 Object.seal(producto)
12
13 //Reescribir un valor
14 producto.disponible = true
15
16 //Sino existe, lo va a añadir
17 producto.imagen = 'imagen.jpg'
18
19 //Eliminar propiedad
20 delete producto.precio
21
22 console.log(producto)
23
```

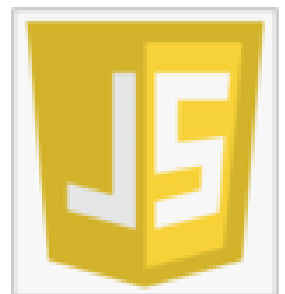
Manipulación de Objetos - seal: Hacer un objeto modificable, pero sin permitir nuevas inserciones

► {nombre: 'Laptop', precio: 800, disponible: false}

► {nombre: 'Laptop', precio: 800, disponible: true}



TIPOS DE DATOS:



Objetos - Destructuring:

```
1 //Objetos - Destructuring de dos o mas objetos
2 const producto = {
3   nombre : 'Laptop',
4   precio : 800,
5   disponible : false
6 }
7
8 const cliente = {
9   nombre : 'Beatriz',
10  premium : true,
11  disponible : false
12 }
13
14 //así marca error
15 const { nombre } = producto
16 const { nombre } = cliente
```

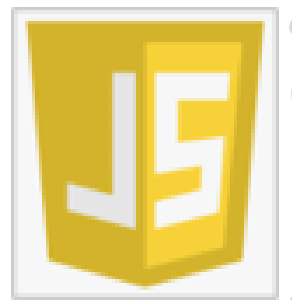
```
1 //Objetos - Destructuring de dos o mas objetos
2 const producto = {
3   nombre : 'Laptop',
4   precio : 800,
5   disponible : false
6 }
7
8 const cliente = {
9   nombre : 'Beatriz',
10  premium : true,
11  disponible : false
12 }
13
14 //así marca error
15 // const { nombre } = producto
16 // const { nombre } = cliente
17
18 const { nombre } = producto
19 const { nombre : nombreCliente } = cliente
20
21 console.log(nombre)
22 console.log(nombreCliente)
```

Laptop

Beatriz



TIPOS DE DATOS:



Objetos - Destructuring: acceder a distintos objetos que tienen elementos con el mismo nombre

```
const cliente = {  
  nombre : 'Beatriz',  
  premium : true,  
  disponible : false,  
  direccion : {  
    calle : 'Calle Larga',  
    numero : 35  
  }  
}  
  
const { nombre } = producto  
const { nombre : nombreCliente, direccion : calle } = cliente //OJO con esto  
console.log(calle)
```

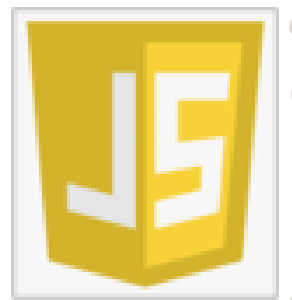
```
▶ {calle: 'Calle Larga', numero: 35}
```

```
const { nombre } = producto  
const { nombre : nombreCliente, direccion : { calle } } = cliente  
console.log(calle)
```

Calle Larga



TIPOS DE DATOS:



Objetos - Destructuring : Unir 2 objetos

```
//Objetos - Destructuring de dos o mas objetos
const producto = {
  nombre : 'Laptop',
  precio : 800,
  disponible : false
}

const cliente = {
  nombre : 'Bea',
  premium : true
}

const carrito = {
  cantidad : 1,
  producto
}

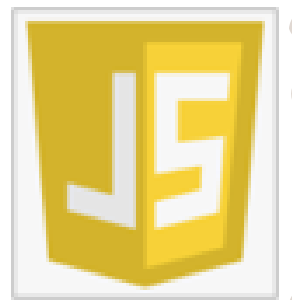
console.log(carrito)
```

```
▼ {cantidad: 1, producto: {...}} ⓘ
  cantidad: 1
  ▶ producto: {nombre: 'Laptop', precio: 800, disponible: false}
  ▶ [[Prototype]]: Object
```

No es así como se debe unir....



TIPOS DE DATOS:



Objetos - Destructuring : Unir 2 objetos

```
//Objetos - Destructuring de dos o mas objetos
const producto = {
  nombre : 'Laptop',
  precio : 800,
  disponible : false
}

const cliente = {
  nombre : 'Bea',
  premium : true
}

const carrito = {
  cantidad : 1,
  ...producto
}

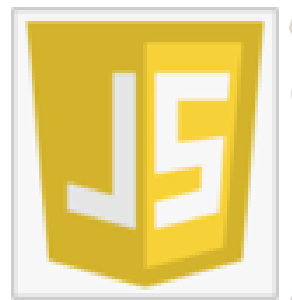
console.log(carrito)
```

```
▼ {cantidad: 1, nombre: 'Laptop', precio: 800, disponible: false}
  cantidad: 1
  disponible: false
  nombre: "Laptop"
  precio: 800
```

Esto ayuda a que la lógica tenga menos código y sea mas limpia, permitiéndome acceder de forma mas directa a los elementos del producto.



TIPOS DE DATOS:



Objetos - Destructuring : Unir 2 objetos

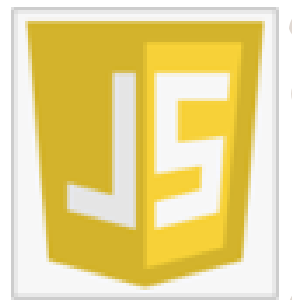
```
const nuevoObjeto = {  
  ...producto, //spread operator  
  ...cliente  
}  
  
console.log(nuevoObjeto)  
  
const nuevoObjeto2 = Object.assign(producto, cliente)  
console.log(nuevoObjeto2)
```

**Instrucciones equivalentes,
pero notar que el nombre se
sobreescribe, ahí hay que
revisar qué se necesita...**

```
js08.js:26  
▶ {nombre: 'Bea', precio: 800, disponible: false, premium: true}  
  
js08.js:29  
▶ {nombre: 'Bea', precio: 800, disponible: false, premium: true}
```



TIPOS DE DATOS:



Template Strings y Concatenación: para evitar la complejidad de unir todo con '+' o ','

```
//Template Strings y Concatenación
const producto = "Tablet de 12 pulgadas"
const precio = 350
const marca = "Lenovo"

console.log(producto + ' $' + precio + ' Dolares, marca: ' + marca)
console.log(`${producto} ${precio} Dolares, marca: ${marca}`) //backtick: teclado español AltGr + Spc + `
```

Tablet de 12 pulgadas \$350 Dolares, marca: Lenovo	<u>js09.js:6</u>
Tablet de 12 pulgadas \$350 Dolares, marca: Lenovo	<u>js09.js:7</u>



LINKS DE UTILIDAD



JS

- ✓ <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- ✓ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object